

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Web Client and Backend Development Framework.

Reporte de Documentación del API con Swagger y OpenAPI

Profesor: M. en C. José Asunción Enríquez Zárate

Alumno: Luis David Martínez Martínez

ld.martinez.117@gmail.com

7CM3

22 de Junio, 2026

Contents

1	Introducción	1
2	Objetivo de la práctica	2
3	Conceptos	3
3.1	<i>OpenAPI y Swagger UI</i>	3
3.2	<i>Springdoc OpenAPI Starter</i>	3
3.3	<i>Anotaciones de OpenAPI</i>	3
4	Descripción del sistema	4
4.1	Módulos Documentados	4
5	Modelo de base de datos	5
6	Desarrollo	6
6.1	<i>Configuración del Proyecto y Dependencias</i>	6
6.2	<i>Clase de Configuración ApiConfig</i>	6
6.3	<i>Integración en Controladores</i>	6
7	Resultados y Evidencias	7
8	Conclusión	9
9	Referencias Bibliográficas	10

List of Figures

1	Panel principal de Swagger UI mostrando todos los controladores documentados.	7
2	Modal donde se ingresan credenciales de autorización.	7
3	Consola de pruebas de Swagger ejecutando una consulta al catálogo de habitaciones.	8

List of Tables

1	Esquema de entidades del sistema en base de datos	5
2	Criterios de Evaluación y Avance de la Tarea 1	9

1 Introducción

La fase de integración y prueba de interfaces de programación de aplicaciones (API) REST representa una de las actividades críticas en el ciclo de vida del desarrollo de software. Tradicionalmente, la documentación de APIs se realizaba mediante documentos de texto estáticos o wikis manuales, lo que propiciaba una rápida desincronización entre la implementación real en el código fuente y las guías de consumo de los clientes web.

Para remediar esta problemática, el ecosistema de desarrollo moderno ha adoptado de forma generalizada la especificación OpenAPI (anteriormente conocida como Swagger). Swagger proporciona una suite de herramientas de código abierto basadas en una especificación común que asiste a los desarrolladores en el diseño, construcción, documentación y consumo de APIs RESTful. A través de interfaces interactivas autogeneradas, los desarrolladores de Front End pueden probar las respuestas del servidor directamente en tiempo real, agilizando el flujo operativo de los equipos de ingeniería de software.

2 Objetivo de la práctica

Integrar y configurar la especificación OpenAPI/Swagger en una aplicación de servicios RESTful desarrollada con el framework Spring Boot y Spring Security. Esto permitirá documentar de forma automatizada y estructurada cada uno de los endpoints de la plataforma (Habitaciones, Reservaciones, Autenticación y Archivos), detallando los tipos de datos de entrada, códigos de respuesta HTTP y políticas de seguridad, facilitando el consumo por parte del cliente Front End.

3 Conceptos

3.1 *OpenAPI y Swagger UI*

OpenAPI es una especificación estandarizada para describir APIs REST sin requerir acceso al código fuente. Swagger UI, por su parte, es una herramienta que consume esta especificación para renderizar una página HTML interactiva y amigable en el navegador, permitiendo a cualquier usuario visualizar e interactuar con los recursos del API de forma inmediata.

3.2 *Springdoc OpenAPI Starter*

Es la librería oficial recomendada para proyectos Spring Boot 3.x y superiores que automatiza la generación de la documentación. Analiza en tiempo de ejecución las anotaciones del controlador de Spring (`@RestController`, `@GetMapping`, etc.) para estructurar el archivo JSON/YAML de OpenAPI.

3.3 *Anotaciones de OpenAPI*

- `@Tag`: Permite agrupar operaciones por recursos lógicos (ej. Habitaciones).
- `@Operation`: Describe el propósito y comportamiento específico de un endpoint.
- `@ApiResponse`: Detalla el código de retorno HTTP (200, 201, 400, 403, 404, 500) y describe su significado en la respuesta.

4 Descripción del sistema

El sistema desarrollado, denominado **ReservaEasy**, consta de una arquitectura desacoplada para la administración hotelera. El módulo de backend en Spring Boot coordina la persistencia en base de datos de habitaciones y reservaciones, mientras que el módulo de seguridad garantiza que solo usuarios registrados accedan a los recursos del hotel.

4.1 Módulos Documentados

1. **Autenticación** (/api/v1/auth): Permite el registro de usuarios y el inicio de sesión.
2. **Habitaciones** (/api/v1/cuartos): Permite listar, crear, modificar y eliminar habitaciones.
3. **Reservaciones** (/api/v1/reservaciones): Registra y cancela reservas, validando la disponibilidad de cuartos en el rango de fechas solicitado.
4. **Archivos** (/apiArchivos/v1/archivos): Proporciona endpoints para subir comprobantes o imágenes a la base de datos y descargarlos.

5 Modelo de base de datos

El modelo de base de datos está compuesto por las entidades principales descritas en la Tabla 1.

Tabla	Descripción	Atributos Clave
rooms	Catálogo de habitaciones	id, tipo, numero, precio, disponible
usuarios	Credenciales de usuarios registrados	id, username, password, email
roles	Permisos del sistema	id, nombre (ROLE_ADMIN, ROLE_USER)
reservaciones	Relación entre usuarios y habitaciones	id, cuarto_id, usuario_id, fecha_inicio
archivo	Soporte digital binario	id_archivo, nombre_archivo, datos_archivo

Table 1: Esquema de entidades del sistema en base de datos

6 Desarrollo

6.1 Configuración del Proyecto y Dependencias

Para integrar la documentación se añadió la dependencia del starter oficial en el archivo `pom.xml`:

```
1 <dependency>
2   <groupId>org.springdoc</groupId>
3   <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
4   <version>2.5.0</version>
5 </dependency>
```

6.2 Clase de Configuración ApiConfig

Se creó una clase de configuración en el paquete `core/config` para personalizar los metadatos globales del API:

```
1 @Configuration
2 public class ApiConfig {
3     @Bean
4     public OpenAPI myOpenAPI() {
5         return new OpenAPI()
6             .info(new Info()
7                 .title("API para la gestion de reservaciones")
8                 .version("1.0.0")
9                 .description("API REST para la gestion de cuartos y reservaciones")
10                .contact(new Contact()
11                    .name("Luis David Martinez")
12                    .email("ld.martinez.117@gmail.com" )))
13     }
14 }
```

6.3 Integración en Controladores

Se agregaron las anotaciones en el controlador de reservaciones:

```
1 @RestController
2 @RequestMapping("/api/v1/reservaciones")
3 @Tag(name = "Modulo de Reservaciones", description = "Endpoints para la gestion del ciclo de vida")
4 public class ReservationController {
5     // ...
6     @PostMapping
7     @Operation(summary = "Crear una nueva reservacion", description = "Crea una reservacion valida")
8     @ApiResponses(value = {
9         @ApiResponse(responseCode = "201", description = "Reservacion creada con exito"),
10        @ApiResponse(responseCode = "400", description = "Fechas invalidas o conflicto de ocupacion")
11    })
12    public ResponseEntity<ReservationDTO> createReservation(@Valid @RequestBody CreateReservationDTO createReservationDTO) {
13        return ResponseEntity.status(HttpStatus.CREATED).body(reservationService.createReservation(createReservationDTO));
14    }
15 }
```

7 Resultados y Evidencias

Tras levantar el backend en el puerto local 8090, accedemos al portal interactivo en la dirección:

`http://localhost:8090/swagger-ui/index.html`

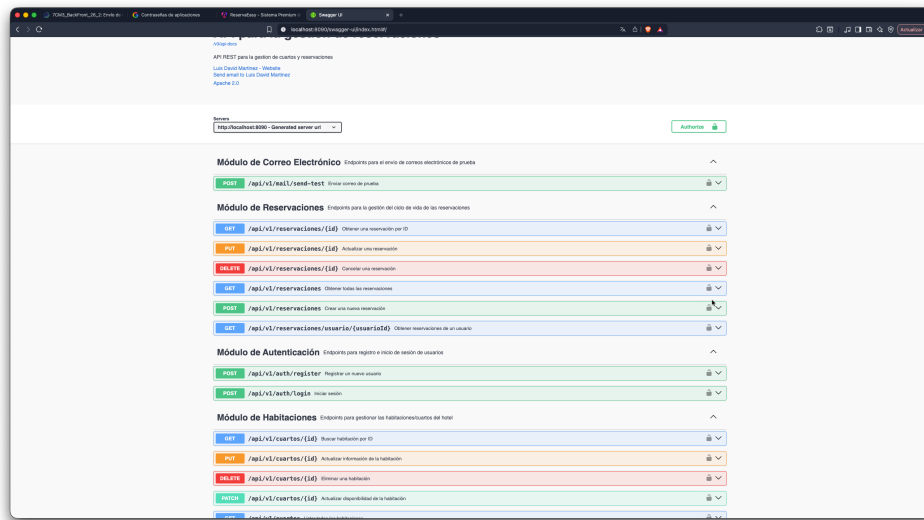


Figure 1: Panel principal de Swagger UI mostrando todos los controladores documentados.

El sistema permite ejecutar pruebas a los endpoints públicos sin credenciales, y solicita credenciales de autenticación básica HTTP para aquellos protegidos por Spring Security.

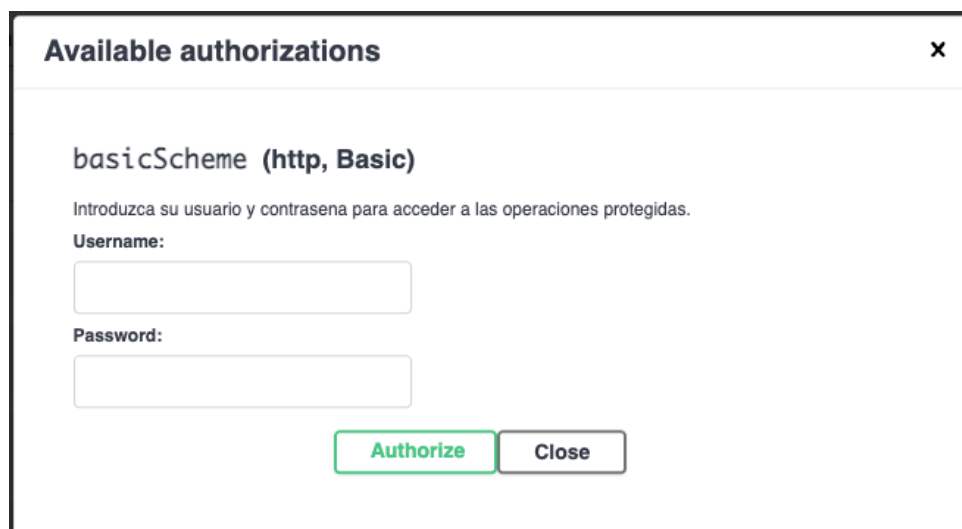


Figure 2: Modal donde se ingresan credenciales de autorización.

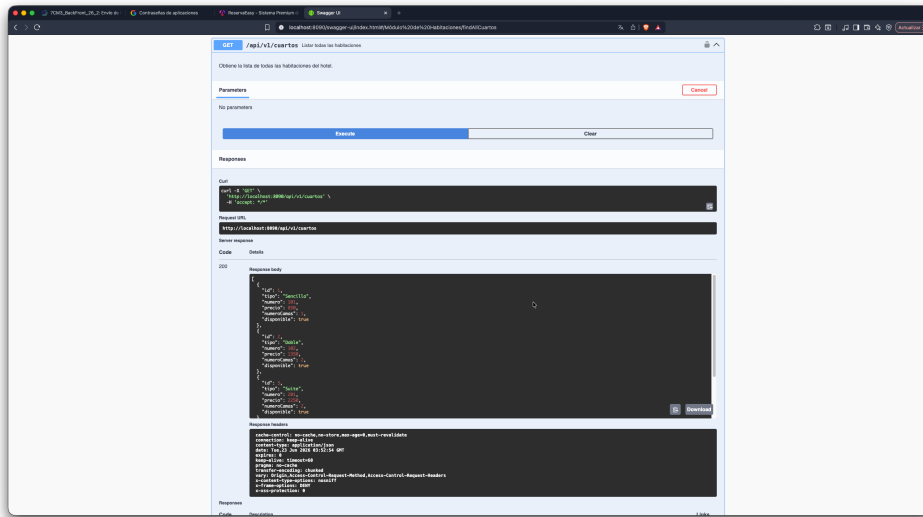


Figure 3: Consola de pruebas de Swagger ejecutando una consulta al catálogo de habitaciones.

8 Conclusión

La documentación autogenerada con Swagger UI reduce la brecha operativa en el desarrollo full stack. Al mantener las definiciones de endpoints acopladas a las anotaciones del código Java, se garantiza que la documentación del API nunca quede obsoleta. Asimismo, proporciona un sandbox óptimo que reduce la dependencia de herramientas de desarrollo locales.

Criterio	Puntos	Realizado
Portada	Sin	100%
Objetivo	30%	30%
Documentación	40%	40%
Calidad de Descripciones	20%	20%
Evidencias	10%	10%

Table 2: Criterios de Evaluación y Avance de la Tarea 1

Luis David Martínez Martínez

9 Referencias Bibliográficas

References

[Springdoc OpenAPI, 2026] Springdoc. *Springdoc OpenAPI documentation helper*. Cloud Guides. Disponible en:
<https://springdoc.org/>

[OpenAPI Initiative, 2026] OpenAPI. *OpenAPI Specification Standard 3.0*. Linux Foundation. Disponible en:
<https://spec.openapis.org/oas/v3.0.3>